

BLG470E - Functional Programming

HW1

Dr. Yusuf H. Şahin

Answer the following questions using Haskell. You may define helper functions when necessary.

Q1 - Prime Function

Write a recursive Haskell function that returns the n th prime number. Assume that the indexing starts from 1. Therefore:

$$\text{nthPrime } 1 = 2$$

Example:

```
nthPrime 1  -- returns 2
nthPrime 2  -- returns 3
nthPrime 5  -- returns 11
```

Your function should have the following type signature:

```
nthPrime :: Int -> Int
```

Q2 - Goldbach's Conjecture

According to Goldbach's conjecture, every even integer greater than 2 can be written as the sum of two prime numbers.

Write a Haskell function that takes an even integer $n > 2$ and returns the larger prime number in such a pair.

If there is more than one possible pair, choose the pair whose smaller prime is as small as possible. Equivalently, return the largest possible prime among the valid Goldbach pairs.

Example:

$$44 = 3 + 41$$

Therefore:

```
goldbachLargerPrime 44  -- returns 41
```

Your function should have the following type signature:

```
goldbachLargerPrime :: Int -> Int
```

Q3 - Y-Combinator

The Y combinator is a fixed-point combinator that allows recursion without explicitly naming a recursive function. In Haskell, a fixed-point combinator can be written as:

```
y :: (a -> a) -> a
y f = f (y f)
```

Using the given fixed-point combinator, define a factorial function without directly writing a recursive call to the factorial function itself.

In other words, you should not write:

```
factorial n = n * factorial (n - 1)
```

Instead, define factorial by using the fixed-point combinator.

Your final function should behave as follows:

```
factorial 0 -- returns 1
factorial 1 -- returns 1
factorial 5 -- returns 120
```

Your function should have the following type signature:

```
factorial :: Integer -> Integer
```

Q4 - PRFs, GRFs

Primitive recursive functions are built from initial functions using composition and primitive recursion. They always define total computable functions.

General recursive functions extend primitive recursive functions by allowing unbounded minimization, which may produce partial functions, i.e., functions that may not terminate for some inputs.

1. The factorial function can be defined using primitive recursion as follows:

$$\begin{aligned}\text{fact}(0) &= 1 \\ \text{fact}(n+1) &= (n+1) \cdot \text{fact}(n)\end{aligned}$$

Write a Haskell function implementing factorial using recursion.

```
factorialPR :: Integer -> Integer
```

2. Consider the following informal definition of an unbounded search function:

$$\mu x. p(x)$$

This function returns the smallest non-negative integer x such that $p(x)$ is true. If no such x exists, the function may never terminate.

Write a Haskell function that implements this idea:

```
minimize :: (Integer -> Bool) -> Integer
```

Example:

```
minimize (\x -> x * x >= 30) -- returns 6
```

3. Explain why the `minimize` function corresponds to general recursion rather than primitive recursion.

Q5 - Function Calls

The following Haskell functions are intentionally written in a non-obvious way. For each GHCi input, write the exact output. If the expression does not terminate, write **Non-terminating**.

```
mystery1 :: Integer -> Integer
mystery1 n
  | n <= 1 = 2
  | even n = n + mystery1 (n `div` 2)
  | otherwise = mystery1 (n - 1) - mystery1 (n - 2)
```

```
mystery2 :: Integer -> Integer -> Integer
mystery2 n acc
  | n == 0 = acc
  | even n = mystery2 (n `div` 2) (acc + n)
  | otherwise = mystery2 (n - 1) (acc * 2)
```

```
alpha :: Integer -> Integer
alpha 0 = 1
alpha n = beta (n - 1) + n
```

```
beta :: Integer -> Integer
beta 0 = 2
beta n = 2 * alpha (n - 1) - beta (n - 1)
```

```
mystery3 :: Integer -> Integer
mystery3 n
  | n > 100 = n - 10
  | otherwise = mystery3 (mystery3 (n + 11))
```

```
mystery4 :: Integer -> Integer
mystery4 n
  | n <= 1 = 1
  | even n = mystery4 (n `div` 2)
            + mystery4 (n `div` 2 - 1)
            + n
  | otherwise = mystery4 (n - 1) - mystery4 (n - 3)
```

```
mystery5 :: Integer -> Integer
mystery5 n
  | n <= 0 = 0
  | n `mod` 3 == 0 = n + mystery5 (step n)
  | otherwise = mystery5 (n - 1) + step n
  where
    step x = x `div` 3
```

```
loopSearch :: Integer -> Integer
loopSearch n = go 0
  where
    go y
      | y * y == n = y
      | otherwise = go (y + 1)
```

What is the result of each GHCi input?

1.
2.
3.
4.
5.
6.
7.
8.
9.
10.